GUIDA OPERATIVA

Claude Registry

Come usare e contribuire alle capability condivise del team — guida pratica con comandi, script e passaggi operativi.

CONTENUTI

Parte 1 — Usare le capability

- Prerequisiti e concetti base
- Installazione rapida con script
- Installazione manuale passo-passo
- Come invocare una capability
- Personalizzare per il proprio progetto
- Aggiornare le capability

Parte 2 — Aggiungere e migliorare capability

- Prerequisiti e ciclo di vita
- Creare una capability con lo script
- Scrivere il system prompt
- Testare localmente
- Aprire una Pull Request
- Rispondere alla review automatica
- Release e pubblicazione
- Migliorare una capability esistente

Repository github.com/luketherose/claude-registry

Aggiornato Aprile 2026

Formato Claude Code subagents (standard ufficiale)

PARTE 1

Usare le capability

Per il developer che vuole iniziare a usare le capability condivise del team nel proprio progetto Claude Code.

1.1 — Concetti base

Prima di iniziare è utile capire cosa stai installando e perché funziona così.

Le **capability** del team sono tecnicamente dei **subagent Claude Code**: file `.md` con frontmatter YAML che Claude Code riconosce quando li trova nella directory `.claude/agents/` di un progetto.

Una volta installati, Claude Code li può usare in due modi:

- **Delega automatica** — Claude legge il campo `description` di ogni subagent e, quando il task dell'utente corrisponde, delega automaticamente al subagent più adatto.
- **Invocazione esplicita** — l'utente digita `/agents` e seleziona manualmente il subagent da usare.

> ℹ **Come funziona la repository**
> Il registry ha due directory: `claude-catalog/` (sorgente, per i contributor) e `claude-marketplace/` (capability approvate, per i consumer). Tu, come consumer, lavori solo con `claude-marketplace/`.

TERMINE	SIGNIFICATO
`subagent`	Il file `.md` — il termine ufficiale di Claude Code
`capability`	Il pacchetto completo (subagent + docs + esempi) — convenzione interna del team
`stable`	Capability testata e approvata — usala in tutti i progetti
`beta`	Capability nuova o in sviluppo — adatta per piloti e contesti non critici

1.2 — Prerequisiti

- ✓ Claude Code installato e funzionante (`claude --version`)
- ✓ Git installato (`git --version`)
- ✓ Accesso al registry su GitHub: `github.com/luketherose/claude-registry`
- ✓ Python 3.x installato (richiesto dallo script di setup)

Clona il registry (una sola volta)

Il registry va clonato una volta sola in una directory stabile sulla tua macchina. Tutti i tuoi progetti poi installeranno le capability da lì.

TERMINALE

```
$ git clone https://github.com/luketherose/claude-registry.git ~/dev/claude-registry
$ cd ~/dev/claude-registry
```

In futuro, per aggiornarlo con le ultime capability:

TERMINALE

```
$ cd ~/dev/claude-registry && git pull origin main
```

1.3 — Installazione rapida (consigliata)

Lo script `setup-capabilities.sh` mostra la lista delle capability disponibili, ti chiede quali installare e aggiorna automaticamente il `settings.json` del tuo progetto.

Script automatico `claude-catalog/scripts/setup-capabilities.sh`

```
# Sintassi:
$ ~/dev/claude-registry/claude-catalog/scripts/setup-capabilities.sh [PROGETTO] [SELEZIONE]

# Caso 1: installa nella directory corrente (interattivo)
$ cd /path/to/mio-progetto
$ ~/dev/claude-registry/claude-catalog/scripts/setup-capabilities.sh

# Caso 2: installa in un progetto specifico (interattivo)
$ ~/dev/claude-registry/claude-catalog/scripts/setup-capabilities.sh /path/to/mio-progetto

# Caso 3: installa tutte le stable senza interazione
$ ~/dev/claude-registry/claude-catalog/scripts/setup-capabilities.sh /path/to/mio-progetto all
```

Lo script mostrerà una lista interattiva come questa:

```
Num  Tier    Nome                        Descrizione
___  ______  __________________________  ____________________________________

  1  stable  software-architect          Use when analyzing or designing system...
  2  stable  functional-analyst          Use when extracting functional requir...
  3  stable  developer-java-spring       Use when writing or reviewing Java/Sp...
  4  beta    technical-analyst           Use when producing a technical analysi...
  ...

Selezione: 1 2 3
```

Al termine lo script:

- Copia i file selezionati in `.claude/agents/` del tuo progetto
- Aggiunge le regole `Agent(name)` al `.claude/settings.json`
- Mostra i prossimi passi

> ✓ **Fatto!**
> Puoi andare direttamente alla sezione 1.5 per verificare l'installazione.

1.4 — Installazione manuale (alternativa)

Se preferisci controllare ogni passaggio a mano, ecco la procedura completa.

1

Crea la directory degli agenti nel progetto

```
$ mkdir -p /path/to/mio-progetto/.claude/agents
```

Claude Code cercherà i subagent esattamente in questa directory.

2

Sfoglia le capability disponibili

```
$ ls ~/dev/claude-registry/claude-marketplace/stable/
developer-java-spring.md  functional-analyst.md  software-architect.md

$ ls ~/dev/claude-registry/claude-marketplace/beta/
api-designer.md  code-reviewer.md  debugger.md  developer-python.md  ...
```

Per leggere la descrizione di una capability prima di installarla:

```
$ head -10 ~/dev/claude-registry/claude-marketplace/stable/software-architect.md
```

3

Copia le capability nel progetto

```
$ cp ~/dev/claude-registry/claude-marketplace/stable/software-architect.md \
   /path/to/mio-progetto/.claude/agents/

$ cp ~/dev/claude-registry/claude-marketplace/stable/developer-java-spring.md \
   /path/to/mio-progetto/.claude/agents/
```

4

Aggiorna .claude/settings.json

Aggiungi le regole di permesso per evitare il prompt di autorizzazione ad ogni delega:

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": [
      "Agent(software-architect)",
      "Agent(developer-java-spring)",
      "Agent(functional-analyst)"
    ]
  }
}
```

⚠ Se `settings.json` non esiste, crea la directory `.claude/` e il file. Se esiste già, aggiungi solo le nuove regole nella lista `allow` esistente.

5

Aggiungi i file a git del tuo progetto

```
$ cd /path/to/mio-progetto
$ git add .claude/
$ git commit -m "add: claude capabilities from registry"
```

In questo modo tutto il team condivide le stesse capability.

1.5 — Verificare l'installazione

Apri Claude Code nella directory del tuo progetto e verifica:

1

Controlla la lista degli agenti

In Claude Code, digita il comando:

```
/agents
```

Dovresti vedere i subagent che hai installato nella lista.

2

Prova una delega automatica

Scrivi un messaggio che dovrebbe triggerare uno dei subagent. Per esempio, con `software-architect` installato:

```
User: Puoi analizzare l'architettura di questo servizio?
```

Claude dovrebbe automaticamente delegare a `software-architect` senza che tu debba specificarlo.

💡 **Se il subagent non appare**

Verifica che il file `.md` sia nella directory `.claude/agents/` (non in sottodirectory), che abbia il frontmatter YAML valido con i campi `name` e `description`, e che il nome nel frontmatter corrisponda al nome del file (senza `.md`).

1.6 — Personalizzare per il proprio progetto

Se una capability del registry è giusta per il tuo dominio ma manca di contesto specifico del progetto (nomi dei moduli, librerie interne, regole di business), puoi creare un override locale leggero.

⚠️ **Non modificare il file originale**

Non editare il file che hai copiato dal registry — si tratta di una copia: le modifiche non arriverebbero al team e renderebbero difficile aggiornare in futuro. Crea invece un file separato.

1

Crea una copia con un nuovo nome

```
$ cp .claude/agents/developer-java-spring.md \
   .claude/agents/developer-java-spring-payments.md
```

2 Cambia il campo `name` nel frontmatter

.CLAUDE/AGENTS/DEVELOPER-JAVA-SPRING-PAYMENTS.MD

```
---
name: developer-java-spring-payments  # ← cambiato
description: >
  Use when writing or reviewing Java/Spring Boot code for the Payments
  service. Extends the base Java/Spring standard with Payments-specific
  conventions.
tools: Read, Edit, Write, Bash, Grep, Glob
model: sonnet
---

# [tutto il contenuto originale rimane invariato]
...

## Project context — Payments Service
- Package root: com.acme.payments
- Usa la libreria interna AuditLogger per tutti gli eventi di audit
- Tutti gli importi monetari usano BigDecimal, mai double o float
- Le transazioni seguono il pattern Saga con Axon Framework
```

Aggiungi una sezione **al fondo** del file con il contesto specifico del progetto. Non riscrivere il prompt esistente.

3 Aggiungi la nuova regola di permesso

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": [
      "Agent(developer-java-spring)",
      "Agent(developer-java-spring-payments)"  // ← aggiunto
    ]
  }
}
```

1.7 — Aggiornare le capability

Non c'è aggiornamento automatico. Per aggiornare una capability nel tuo progetto:

TERMINALE

```
# 1. Aggiorna il registry locale
$ cd ~/dev/claude-registry && git pull origin main

# 2. Controlla le novità nel changelog
$ cat ~/dev/claude-registry/claude-catalog/CHANGELOG.md | head -40

# 3. Copia la versione aggiornata nel tuo progetto
$ cp ~/dev/claude-registry/claude-marketplace/stable/software-architect.md \
   /path/to/mio-progetto/.claude/agents/software-architect.md

# 4. Rivedi il diff PRIMA di committare
$ cd /path/to/mio-progetto && git diff .claude/agents/software-architect.md

# 5. Commit
$ git add .claude/agents/ && git commit -m "update: software-architect@1.1.0"
```

Aggiornamenti MAJOR

Se la versione fa un salto MAJOR (es. 1.x.x → 2.0.0), leggi il CHANGELOG prima di aggiornare: il campo `description` o `name` potrebbe essere cambiato, il che modifica come Claude decide di delegare. Se hai un override locale basato su questo file, aggiorna anche quello.

PARTE 2

Aggiungere e migliorare capability

Per il membro del team che vuole contribuire al registry: creare nuove capability o migliorare quelle esistenti.

2.1 — Prerequisiti

- ✓ Git installato e configurato (`git config user.email`)
- ✓ GitHub CLI installato: `gh --version` (oppure usa la UI GitHub)
- ✓ Autenticato con GitHub CLI: `gh auth status`
- ✓ Registry clonato in locale: `cd ~/dev/claude-registry`
- ✓ Letto `claude-catalog/how-to-write-a-capability.md`

Setup iniziale (una sola volta)

```
$ git clone https://github.com/luketherose/claude-registry.git ~/dev/claude-registry
$ cd ~/dev/claude-registry
$ cat claude-catalog/how-to-write-a-capability.md  # leggi prima di procedere
```

2.2 — Il ciclo di vita di una capability

Prima di iniziare, capisci dove si inserisce il tuo contributo nel processo:

FASE	DOVE	CHI FA COSA
Draft	Branch `add/nome`	Autore scrive il subagent, esempi, eval
Review	Pull Request su GitHub	GitHub Actions valida automaticamente; reviewer legge

Approved	Merge su `main`	Reviewer approva manualmente
Released	Git tag `name@x.y.z`	Maintainer applica il tag
Published	`claude-marketplace/beta/` o `stable/`	Maintainer esegue lo script di publish

> **Nota sulla branch protection**
> Il branch `main` è protetto. Non puoi fare push diretto.
> **Ogni modifica richiede una Pull Request**
> che deve: (1) passare la validazione automatica di GitHub Actions e (2) ricevere almeno 1 approvazione manuale dal maintainer.

2.3 — Creare una nuova capability

Lo script `new-capability.sh` crea automaticamente il branch git e tutti i file di scaffolding con i placeholder corretti, così non devi ricordare la struttura a memoria.

Script automatico `claude-catalog/scripts/new-capability.sh`

```
# Vai nella directory del registry
$ cd ~/dev/claude-registry

# Esegui lo script (chiede il nome interattivamente)
$ ./claude-catalog/scripts/new-capability.sh

# Oppure passa il nome direttamente
$ ./claude-catalog/scripts/new-capability.sh api-security-reviewer
```

Lo script crea automaticamente:

FILE CREATO	CONTENUTO
`claude-catalog/agents/{nome}.md`	Subagent con frontmatter e placeholder per il system prompt
`claude-catalog/examples/{nome}-example.md`	Template per esempi d'uso
`claude-catalog/evals/{nome}-eval.md`	Template per scenari di valutazione
`CHANGELOG.md`	Voce `[Unreleased]` aggiornata

E crea il branch git `add/{nome}` pronto per il lavoro.

2.4 — Scrivere il system prompt

Il file più importante da completare è `agents/{nome}.md`. Ecco le regole fondamentali.

Struttura del file subagent

CLAUDE-CATALOG/AGENTS/MIA-CAPABILITY.MD

```
---
name: mia-capability          # deve corrispondere al nome file (senza .md)
description: >
  Use when [trigger preciso].    # inizia sempre con "Use when"
  Handles [lista task specifici].
  Does NOT [cosa NON fa].
tools: Read, Grep, Glob         # set minimo necessario
model: sonnet                  # default per la maggior parte dei casi
color: blue
---

## Role
You are a senior [ruolo] specializzato in [dominio]...

## What you always do
- [comportamento obbligatorio 1]
- [comportamento obbligatorio 2]

## What you never do
- [proibizione esplicita 1]

## Output format
[definisci esattamente il formato dell'output: sezioni, tabelle, campi]

## Quality self-check before responding
[cosa il subagent deve verificare sul proprio output prima di rispondere]
```

Regole per il campo `description`

Il campo `description` è il più critico — determina quando Claude delegherà automaticamente a questa capability.

❌ SBAGLIATO	✓ CORRETTO
Helps with software architecture.	*Use when analyzing or designing system architecture, evaluating technology choices, or writing ADRs.*

Works with Spring Boot applications.	*Use when writing, reviewing, or refactoring Java/Spring Boot code. Produces production-ready code with clean architecture.*

Tool permissions — usa il minimo necessario

PROFILO	TOOLS	QUANDO
Solo analisi	`Read, Grep, Glob`	Analisi senza output scritto
Analisi + report	`Read, Grep, Glob, Write`	Produce documenti (ADR, report)
Developer	`Read, Edit, Write, Bash, Grep, Glob`	Scrive e modifica codice
Research	`Read, Grep, Glob, WebFetch`	Ricerca architetturale con lookup web

💡 **Sii opinionated**

Un system prompt vago produce output vago. Una buona capability ha standard espliciti, format di output definiti, e sa cosa non deve fare. Leggi i file completi di `software-architect.md` e `developer-java-spring.md` come riferimento.

2.5 — Testare localmente prima della PR

Non aprire una PR senza aver testato la capability in almeno un caso reale.

1 **Copia il file in un progetto di test**

```
$ mkdir -p /tmp/test-project/.claude/agents
$ cp claude-catalog/agents/mia-capability.md \
   /tmp/test-project/.claude/agents/
```

2 **Apri Claude Code nella directory di test**

```
$ cd /tmp/test-project
$ claude
```

3 Esegui i tuoi scenari di eval

Apri il file `evals/mia-capability-eval.md` e prova ogni scenario descritto. Verifica che l'output rispetti le caratteristiche attese.

4 Verifica con lo script di validazione

Esegui lo stesso validatore che usa GitHub Actions:

```
$ cd ~/dev/claude-registry
$ python3 .github/scripts/validate_catalog.py \
   --changed-files claude-catalog/agents/mia-capability.md
```

Zero errori prima di aprire la PR.

2.6 — Aprire la Pull Request

1 Commit dei file

```
$ cd ~/dev/claude-registry
$ git add claude-catalog/agents/mia-capability.md
$ git add claude-catalog/examples/mia-capability-example.md
$ git add claude-catalog/evals/mia-capability-eval.md
$ git add claude-catalog/CHANGELOG.md
$ git commit -m "add: mia-capability (draft)"
```

2 Push del branch

```
$ git push origin add/mia-capability
```

3 **Crea la Pull Request**

Con GitHub CLI:

```
$ gh pr create \
  --title "add: mia-capability" \
  --body "Aggiunge la capability [nome]. Serve per [use case]. Testata su [progetto]."
```

Oppure apri la URL che appare dopo il push nella UI di GitHub.

2.7 — Rispondere alla review automatica

Entro pochi secondi dall'apertura della PR, GitHub Actions esegue la validazione e posta un commento con il risultato. Non serve aspettare il reviewer prima di leggere questo feedback.

Caso A — Nessun errore

Il commento mostra:

```
## Catalog Validation Report
### ✅ All checks passed
No errors or warnings. This PR is ready for manual review.
```

Puoi taggare il reviewer per l'approvazione manuale.

Caso B — Errori bloccanti

Il commento mostra una lista di errori come:

```
### ❌ Errors — must fix before merge (2)

- `claude-catalog/agents/mia-capability.md` — `name: mia_capability` must be
  lowercase letters, digits, and hyphens only.
- `claude-catalog/agents/mia-capability.md` — `description` should start
  with "Use when", "Use for", or "Use to".
```

Per correggere: edita il file, fai un nuovo commit e push. GitHub Actions ri-eseguirà la validazione e aggiornerà il commento:

```
$ git add claude-catalog/agents/mia-capability.md
$ git commit -m "fix: naming e description field"
$ git push origin add/mia-capability
```

Caso C — Solo warnings

La PR non è bloccata, ma il reviewer potrebbe chiederti di risolvere i warning prima di approvare. Warnings tipici:

- Mancano gli esempi in `examples/`
- Mancano gli eval in `evals/`
- La `description` non inizia con "Use when"

> **Il commento si aggiorna automaticamente**
> Non serve cercare il commento di validazione precedente — GitHub Actions lo aggiorna in-place ad ogni push. Guarda sempre il commento più recente.

2.8 — Release e pubblicazione

Questa fase è responsabilità del **maintainer** del registry. Se sei l'autore, puoi fare il maintainer stesso o chiedere a chi gestisce il repository. Ecco come funziona.

1. **Dopo il merge: aggiorna CHANGELOG.md**

   Sposta la voce da `[Unreleased]` a una sezione con versione e data:

   CHANGELOG.MD

```
## [mia-capability@0.1.0] - 2026-05-01

### Added
- Prima release come capability beta
```

2 **Applica il git tag**

```
$ git checkout main && git pull origin main
$ git tag mia-capability@0.1.0
$ git push origin mia-capability@0.1.0
```

3 **Esegui lo script di publish**

Lo script copia il file nel marketplace, aggiorna `catalog.json` e crea un commit.

Script di publish `claude-marketplace/scripts/publish.sh`

```
# Sintassi: ./publish.sh [nome] [versione] [tier]

# Prima release: vai in beta
$ ./claude-marketplace/scripts/publish.sh mia-capability 0.1.0 beta

# Promozione a stable (dopo test in produzione)
$ ./claude-marketplace/scripts/publish.sh mia-capability 1.0.0 stable
```

4 **Push e comunicazione**

```
$ git push origin main
```

Avvisa il team su Slack/Teams della nuova capability disponibile con il nome e una riga di descrizione.

Promozione beta → stable

Una capability passa da `beta` a `stable` quando:

- È usata in almeno 2 progetti
- Non ha avuto issue critici per almeno 30 giorni
- Supera tutti gli scenari in `evals/`

2.9 — Migliorare una capability esistente

Se vuoi correggere un bug nel prompt, aggiungere un'istruzione mancante, o migliorare il formato di output di una capability già pubblicata, il processo è quasi identico ma con alcune considerazioni in più sul versioning.

1 Crea un branch di tipo `update/`

```
$ cd ~/dev/claude-registry && git checkout main && git pull
$ git checkout -b update/software-architect
```

2 Edita il file in `claude-catalog/agents/`

```
$ $EDITOR claude-catalog/agents/software-architect.md
```

Modifica **solo** `claude-catalog/agents/` — mai direttamente i file in `claude-marketplace/`.

3 Determina il tipo di modifica e la versione

TIPO DI MODIFICA	BUMP VERSIONE
Bug fix nel prompt, comportamento invariato	PATCH: 1.0.0 → 1.0.1
Nuovo comportamento o nuovo formato di output	MINOR: 1.0.0 → 1.1.0
Cambio al campo `name` o `description`	MAJOR: 1.0.0 → 2.0.0
Rimozione di un tool dalla lista `tools`	MAJOR: 1.0.0 → 2.0.0

4 Aggiorna CHANGELOG.md e apri la PR

```
$ git add claude-catalog/agents/software-architect.md claude-catalog/CHANGELOG.md
$ git commit -m "update: software-architect — aggiunge analisi costi"
$ git push origin update/software-architect
$ gh pr create --title "update: software-architect — analisi dimensione costi"
```

Il processo di review e publish è identico a quello di una nuova capability.

> ⚠ **Attenzione ai MAJOR changes**
> Un cambio MAJOR richiede una migration note nel CHANGELOG e una comunicazione proattiva al team, perché i progetti che usano `Agent(software-architect)` in `settings.json` potrebbero doversi aggiornare se il nome cambia.